

AI Resume & Career Advisor

Tech Stack Python, FastAPI, PyPDF2, Google Gemini API (gemini-3.6-flash), HTML5, CSS3, JavaScript
Repository github.com/parvgoyel6/AI-Career-Advisor

1. PROBLEM STATEMENT

Job seekers and fresh college graduates frequently struggle to tailor their resumes effectively to modern corporate expectations and pass automated Applicant Tracking Systems (ATS). Generic resume builders lack targeted, context-aware insights regarding specific Job Descriptions (JDs), leaving candidates unaware of critical skill discrepancies, deprived of customized interview preparation roadmaps, and without structured upskilling timelines.

Project Goal: To build an automated, intelligent, end-to-end career guidance platform that evaluates a candidate's resume directly against an uploaded target job description. The platform calculates a contextual match score, pinpoints technical and soft skill gaps, and synthesizes a tailored interview preparation roadmap along with a pragmatic, 3-month week-by-week learning syllabus.

2. SYSTEM ARCHITECTURE

The platform implements a modular, decoupled four-tier architecture designed for optimal throughput, zero disk persistence of sensitive candidate data, and reliable LLM evaluation:

1. Client / User Interface: Web browser loads responsive dual-file upload interface (Resume & JD PDFs).

▼ Asynchronous Multipart POST (/analyze)

2. Backend & In-Memory Parsing Layer: FastAPI consumes PDF streams directly via `io.BytesIO` and `PyPDF2`; text extracted without disk writes.

▼ Raw Text Streams & Context Grounding

3. Prompt Construction & Agent Layer: System dynamically wraps parsed texts into a persona-driven, schematized prompt.

▼ LLM Request via Google GenAI SDK

4. LLM Core (Google Gemini 3.6 Flash): Evaluates fit, generates deterministic match score, gap analysis, and 3-month roadmap.

▼ Structured JSON Payload

5. Output & Presentation Layer: Frontend dynamically renders scores, missing skill tags, and week-by-week milestones.

Detailed Layer Breakdown

Client / UI Layer

Lightweight, modern web UI built with semantic HTML5, clean CSS animations, and Vanilla JavaScript. Supports drag-and-drop dual document selection and asynchronous updates via the Fetch API.

RAG & Extraction Layer	Asynchronous FastAPI service operating strictly in-memory. Incoming PDF byte arrays are digested using PyPDF2 without ever writing user resumes to disk, safeguarding confidentiality and eliminating file-system cleanup debt.
LLM Agent Layer	The extracted plain texts are synthesized into an explicit prompt scaffold executed by Google's latest gemini-3.6-flash model via the new Google GenAI SDK.
Output & Rendering Layer	Parsed LLM outputs are returned as structured payloads and transformed into interactive, high-readability UI cards with visual score meters.

3. PROMPT DESIGN & ENGINEERING

To ensure deterministic, hallucination-resistant, and parseable evaluations, the system leverages role-prompting combined with strict structural schematization.

```
You are an expert Corporate Career Advisor and Resume Reviewer. Analyze the provided Resume against the Job Description. Job Description: {jd_text} Candidate Resume: {resume_text} Provide the output in exactly this structure: 1. Resume Score: (out of 100 based on fit) 2. Missing Skills: (Bullet points of what is lacking) 3. Interview Roadmap: (Key concepts to study for this specific role) 4. 3-Month Learning Plan: (A brief week-by-week guide to master the missing skills)
```

Key Prompt Engineering Techniques

- **Persona Framing:** Setting the persona as a "Corporate Career Advisor and Resume Reviewer" establishes an authoritative, constructive, and realistic corporate hiring lens.
- **Context Grounding (RAG Simulation):** By grounding the evaluation strictly between {jd_text} and {resume_text}, the agent's attention window remains bounded, virtually eliminating hallucinations.
- **Output Schematization:** Enforcing a fixed four-part structure guarantees consistent output formatting for dependable regex/JSON parsing and intuitive UI rendering.

4. EVALUATION & RESULTS

The system was benchmarked across distinct candidate profiles to assess alignment precision, gap detection capability, and roadmap applicability:

Test Case	Candidate Profile	Target Job Description	Match Score	Key Identified Gaps
Case 1: Standard Fit	Fresher (Java, SQL, OOPs)	Junior Java Developer	78 / 100	Spring Boot, Docker, CI/CD pipelines
Case 2: Role Pivot	Data Analyst (Python, Excel)	Machine Learning Engineer	54 / 100	Model Deployment (FastAPI), PyTorch, MLOps
Case 3: High Alignment	Full-Stack Dev (React, Node)	Frontend Engineer	88 / 100	System Design, Unit Testing (Jest)

Key Observations: The system demonstrated strong discrimination between transferable background skills and mandatory core qualifications. In Case 2 (Role Pivot), the model successfully credited foundational Python skills while appropriately penalizing the candidate for missing production MLOps experience, pairing the lower score with a dedicated remedial study sequence.

5. CHALLENGES & SOLUTIONS

Challenge 1: Handling In-Memory PDF Uploads

Problem: Persisting uploaded candidate resumes to the server filesystem presented data privacy vulnerabilities, disk bloat, and file-cleanup concurrency issues.

Solution: Streamlined digestion directly using Python's `io.BytesIO` buffers and `PyPDF2.PdfReader`, keeping all textual extraction purely transient in RAM.

Challenge 2: API Deprecations & Model Transition

Problem: Legacy Google Gemini SDK endpoints experienced deprecation, resulting in runtime 404 client failures.

Solution: Upgraded the backend integration to the official `google-generativeai` SDK, binding requests to the high-performance, cost-effective `gemini-3.6-flash` engine.

Challenge 3: Secret Leakage & GitHub Push Protection

Problem: GitHub automated push protection blocked repository commits containing unmasked API credentials during prototype development.

Solution: Strictly segregated credentials into localized `.env` files managed via `python-dotenv`, enforced strict `.gitignore` rules, and sanitized the commit history.

6. FUTURE ENHANCEMENTS

- **Vector Database Integration:** Incorporate ChromaDB or FAISS with recursive semantic chunking to support extensive multi-page portfolios, patents, and academic publications.
- **Direct Export Engine:** Provide one-click PDF and DOCX generation to allow candidates to export and print their tailored 3-month upskilling syllabus.
- **Interactive AI Mock Interviewer:** Extend the agent pipeline with real-time audio/interactive chat capabilities to conduct simulated technical and behavioral interviews based on the identified weaknesses.